SmartUzhavan

v4.0

MERN Stack Backend

Complete Implementation Guide

Farm Management System

Generated: May 05, 2026

Zero-Cost, Production-Ready Architecture

Table of Contents

1. Executive Summary

SmartUzhavan v4 is a complete farm management system backend built on the MERN stack (MongoDB, Express, React, Node.js). This document provides a comprehensive implementation guide for building a production-ready backend with zero infrastructure costs.

Key Highlights:
• Complete REST API with 18 endpoints
• Real-time multi-user synchronization via Socket.io
• Comprehensive audit logging and change history
• Server-side reporting and CSV/Excel export
• Session-based authentication for rural users
• Scalable to 100+ concurrent users
• Zero monthly infrastructure costs ($0/month)
• Production deployment ready (Railway/Render)
• Estimated implementation: 27.5 hours over 5 weeks

2. Project Overview & Requirements

2.1 Current State

Backend: Node.js server exists (PDF generation only) | Database: None (localStorage only) | Real-time: Not implemented | Auth: Basic login without persistence

2.2 Target State

Full MERN stack with MongoDB persistence, real-time Socket.io, comprehensive audit logging, multi-admin support, and production deployment

2.3 Requirements

Requirement	Status	Priority
Hybrid: localStorage + MongoDB	✓	High
Session-based Authentication	✓	High
Real-time Multi-user Sync	✓	High
Audit Logging (who/what/when)	✓	High
Change History & Restore	✓	Medium
Server-side Reporting	✓	Medium
CSV/Excel Export	✓	Medium

Full-text Search	✓	Low
Scale to 100+ users	✓	High
Zero Infrastructure Costs	✓	High

3. Technology Stack & Rationale

Component	Technology	Reasoning	Cost
Runtime	Node.js 18 LTS	Stable, performant	Free
Framework	Express.js 4.x	Lightweight, flexible	Free
Database	MongoDB Atlas	Free 512MB, JSON-native	Free
Sessions	express-session	Stateful, works offline	Free
Real-time	Socket.io 4.x	Multi-user, handles reconnects	Free
Auth	bcryptjs	Industry standard hashing	Free
Validation	express-validator	Middleware validation	Free
Hosting	Railway/Render	Free tier, auto-deploy	Free
Deployment	GitHub Actions	CI/CD automation	Free

Total Monthly Cost: $0 (All free tiers, scalable when needed)

4. Architecture Overview

4.1 Hybrid Sync Architecture

Local Layer (Browser): localStorage for offline-first UX with immediate reads/writes
Cloud Layer (MongoDB): Source of truth, persists all data
Sync Mechanism: Periodic sync when online, timestamp-based conflict resolution
Benefits: Works offline, responsive UI, auto-recovery when back online

4.2 Authentication Flow

1. User enters credentials → POST /api/auth/login
2. Validate against MongoDB via bcrypt comparison
3. express-session creates encrypted httpOnly cookie
4. Session persisted in MongoDB via connect-mongo
5. Subsequent requests include session cookie (auto-verified by middleware)
6. Role-based access control (admin, super_admin, operator)

4.3 Real-time Synchronization

Socket.io enables live updates when multiple admins online:
→ Admin A creates farmer → farmer:created event fired
→ Server broadcasts to all connected Socket.io clients
→ Admin B's browser updates UI in real-time
→ localStorage auto-syncs in background
→ Handles reconnections gracefully

5. Database Design (MongoDB)

5.1 Collection: users

Purpose: Admin authentication and authorization
Fields: _id, username (unique), password (hashed), email (unique), role (admin/super_admin/operator), lastLogin, createdAt, isDeleted
Indexes: username, email, isDeleted

5.2 Collection: farmers

Purpose: Core entity with audit trail
Fields: _id, name, phone, village, landArea, crops[], soilType, metadata{}, createdBy, updatedBy, createdAt, updatedAt, isDeleted

Indexes: name (text), village, createdAt, isDeleted, crops

5.3 Collection: auditLogs

Purpose: Track all user actions for compliance
Fields: _id, userId, action (CREATE/UPDATE/DELETE), entity, entityId, beforeData, afterData, ipAddress, timestamp
Indexes: userId+timestamp, entity+entityId+timestamp

5.4 Collection: changeHistory

Purpose: Version control with restore capability
Fields: _id, documentId, documentType, version, data{}, changedBy, changedAt
Indexes: documentId+version, documentId+documentType

6. API Specification (18 Endpoints)

6.1 Authentication Endpoints (4)

Endpoint	Method	Purpose
/api/auth/register	POST	Register new admin (super_admin only)
/api/auth/login	POST	Authenticate and create session
/api/auth/profile	GET	Get current user profile
/api/auth/logout	POST	Destroy session

6.2 Farmer CRUD Endpoints (5)

Endpoint	Method	Purpose
/api/farmers	GET	List with pagination, filtering, search
/api/farmers	POST	Create new farmer
/api/farmers/:id	GET	Get single farmer + history
/api/farmers/:id	PUT	Update farmer
/api/farmers/:id	DELETE	Soft delete farmer

6.3 History & Search Endpoints (3)

/api/farmers/:id/history (GET) - Get version history
/api/farmers/:id/restore/:version (POST) - Restore previous version
/api/farmers/search (POST) - Full-text search

6.4 Reporting Endpoints (3)

/api/reports/summary (GET) - Dashboard statistics
/api/reports/farmers/export (GET) - CSV/Excel export
/api/reports/audit-log (GET) - Audit trail viewer

6.5 WebSocket Events (3)

farmer:created - Broadcast when new farmer added
farmer:updated - Broadcast when farmer modified
farmer:deleted - Broadcast when farmer soft deleted

7. Implementation Phases (27.5 Hours)

Phase	Duration	Tasks	Deliverable
1: Foundation	6 hours	Setup, Auth, CRUD	REST API + Auth
2: Audit & History	4 hours	Logging, Versioning	Tracking System
3: Real-time	5 hours	Socket.io, Sync	Multi-user Sync
4: Reporting	5 hours	Stats, Export, Search	Analytics
5: Deployment	7.5 hours	Deploy, Test, Monitor	Live System

7.1 Phase 1: Foundation (Week 1-2, 6 hours)

Tasks:
■ Project initialization (npm, package.json, dependencies)
■ MongoDB Atlas setup and connection
■ Create 4 Mongoose models with validation
■ Implement authentication routes (register, login, logout)
■ Implement Farmer CRUD endpoints
■ Add input validation to all routes
■ Create Postman collection and test all endpoints
Deliverable: Working REST API with basic auth

7.2 Phase 2: Audit & History (Week 3, 4 hours)

Tasks:
■ Create audit logging middleware
■ Log all mutations (CREATE, UPDATE, DELETE) with before/after data
■ Implement change history tracking
■ Create history API endpoints (GET history, restore version)
■ Verify soft deletes working correctly
Deliverable: Complete audit trail and version control

7.3 Phase 3: Real-time (Week 4, 5 hours)

Tasks:
■ Install and configure Socket.io
■ Implement session-based Socket.io authentication
■ Create farmer:created, farmer:updated, farmer:deleted events
■ Broadcast events to all connected clients
■ Implement browser-server sync mechanism
■ Handle offline/online transitions gracefully
Deliverable: Real-time multi-user synchronization

7.4 Phase 4: Reporting (Week 5, 5 hours)

Tasks:

- Implement dashboard statistics endpoint
- Create server-side CSV/Excel export with streaming
- Add full-text search functionality
- Implement audit log viewer with filtering
- Test with large datasets (5000+ records)

Deliverable: Business intelligence and export capabilities

7.5 Phase 5: Deployment (Week 6, 7.5 hours)

Tasks:

- Configure environment variables for production
- Choose deployment platform (Railway or Render)
- Connect GitHub and setup auto-deployment
- Test backend from production URL
- Integration testing (login, CRUD, sync, export)
- Performance and load testing
- Security review and hardening
- Create deployment and runbook documentation

Deliverable: Production-ready live system

8. Authentication & Security

8.1 Session-Based Authentication

Why Session-Based? Works offline, easier for rural internet users, no token management complexity
Implementation: express-session with MongoDB store (connect-mongo)
Security: httpOnly cookies (prevents XSS), Secure flag (HTTPS only), SameSite=Lax (CSRF protection)
Timeout: 24 hours inactivity auto-logout

8.2 Password Security

Algorithm: bcryptjs with 10 salt rounds
Storage: Never store plain text, always hash
Validation: Minimum 8 characters, requires uppercase, lowercase, number, special char
Never Logged: Exclude passwords from all logs

8.3 Role-Based Access Control

super_admin: Can register users, delete records, view audit logs
admin: Can create/edit/delete farmers, export data
operator: Can view and create farmers, limited export
Implementation: Middleware checks user.role on protected routes

8.4 Input Validation

Validation Library: express-validator
Scope: All POST/PUT endpoints validate inputs before database
Rules: Type checking, length limits, regex patterns, enum validation
Response: 400 Bad Request with field-level error messages

8.5 Rate Limiting

Login Attempts: 5 per 15 minutes per IP
General API: 100 requests per minute per IP
Export: 5 per hour per user
Implementation: express-rate-limit middleware

9. Real-time Synchronization Strategy

9.1 Conflict Resolution

Strategy: Last-Write-Wins with timestamps
Logic: If DB timestamp > local timestamp, use DB version (server is source of truth)
Offline Queue: Queue updates while offline, flush on reconnection
User Notification: Alert if local changes overwritten by another user's changes

9.2 Socket.io Connection Handling

Authentication: Session cookie validated on connection
Reconnection: Auto-reconnect with exponential backoff
Offline Detection: Client detects no internet, queues operations
Recovery: On reconnect, sync queued operations to server
Heartbeat: Ping/pong every 30 seconds to detect dead connections

9.3 Data Consistency

Database: MongoDB is source of truth
localStorage: Cache layer for offline support
Browser Session: Syncs with DB on every write
Transactions: MongoDB transactions ensure atomicity for multi-step operations

10. Reporting & Analytics

10.1 Dashboard Statistics

Metrics: Total farmers, active farmers, total land area, average land area per farmer
Distributions: Top villages, crop distribution, soil type distribution
Activity: Records created/updated/deleted in date range
Implementation: MongoDB aggregation pipeline for performance

10.2 CSV/Excel Export

Scope: Export all or filtered farmers with custom field selection
Performance: Server-side streaming (no client-side memory limits)
Formats: CSV and XLSX supported
Includes: Metadata columns, created/updated dates, created by user
Limit: Handles 10,000+ records efficiently

10.3 Full-Text Search

Indexed Fields: name, phone, village
Implementation: MongoDB text indexes with relevance scoring
Results: Ranked by relevance, highlights matching terms
Performance: <200ms for typical search across 1000 farmers

10.4 Audit Log Viewer

Filters: By user, action (CREATE/UPDATE/DELETE), date range
Display: Who changed what, when, and what changed (before/after)
Pagination: 50 records per page, sortable by timestamp
Compliance: Audit-ready export for regulatory requirements

11. Deployment Guide

11.1 Railway.app Deployment (Recommended)

Step 1: Create Railway account and connect GitHub
Step 2: Select your backend repository
Step 3: Add environment variables (MONGODB_URI, SESSION_SECRET, etc.)
Step 4: Railway auto-deploys on git push
Step 5: Get production URL (e.g., https://smartuzhavan-backend-production.railway.app)
Free Tier: $5 credit/month (sufficient for hobby projects)

Deployment Time: 5 minutes for initial setup, 30-60 sec per redeploy

11.2 Render.com Deployment (Alternative)

Step 1: Create Render account and connect GitHub
Step 2: Create 'Web Service' and select repository
Step 3: Configure build/start commands and environment variables
Step 4: Deploy and monitor logs
Free Tier: Hobby plan (spins down after 15 min inactivity)
Deployment Time: 2-3 minutes for initial setup

11.3 Environment Variables

NODE_ENV: production
PORT: 5000 (or platform-provided)
MONGODB_URI: From MongoDB Atlas
SESSION_SECRET: 32+ random characters (use: node -e "console.log(require('crypto').randomBytes(32).toString('hex'))")
FRONTEND_URL: Your React app's production URL
LOG_LEVEL: info (production)

11.4 Post-Deployment Checklist

■ Test /health endpoint returns 200
■ Database connection working (check logs)
■ Can login with test credentials
■ Create farmer and verify in DB
■ Test Socket.io real-time events
■ CSV export working
■ Audit logs being created
■ No errors in logs for 24 hours
■ Response times acceptable (<500ms)

12. Monitoring & Maintenance

12.1 Daily Tasks

- Check for errors in backend logs
- Monitor MongoDB storage usage (alert if >500MB)
- Verify no stuck processes
- Spot-check API response times

12.2 Weekly Tasks

- Review error logs for patterns
- Test critical user flows (login → create farmer → export)
- Check database query performance
- Review audit logs for suspicious activity

12.3 Monthly Tasks

- MongoDB backup and test recovery
- Security review (update dependencies, check for vulnerabilities)
- Load test with production-like data volume
- Review and optimize slow queries

13. Testing Strategy

13.1 Unit Tests

- User model validation
- Farmer model validation
- Password hashing
- Soft delete filtering

13.2 Integration Tests

- Register → Login → Profile → Logout flow
- Create farmer → triggers audit log
- Update farmer → creates change history
- Delete farmer → soft delete only
- Socket.io farmer:created broadcast
- Search functionality

13.3 Load Testing

- 100 concurrent users
- 1000+ farmer records
- Pagination with large datasets
- CSV export with 5000+ records

13.4 Security Testing

- SQL injection prevention
- XSS protection
- CSRF protection
- Session hijacking prevention
- Password hashing verification
- Rate limiting under attack

14. Troubleshooting Guide

14.1 Common Issues

Issue	Solution
MongoDB connection timeout	Check MONGODB_URI, whitelist IP in Atlas
Session not persisting	Verify connect-mongo connection, restart server
Socket.io not connecting	Check CORS config, frontend listening for events
Slow database queries	Check indexes, analyze query with explain()
Memory errors in production	Check for memory leaks, increase plan
CORS errors	Verify FRONTEND_URL matches exact domain
Build fails on deployment	Check package.json, ensure all deps listed

15. Success Criteria & KPIs

15.1 Phase 1 Complete When

✓ All 8 CRUD endpoints working in Postman
✓ Login/logout working
✓ Session persistence verified
✓ Input validation on all endpoints
✓ Postman collection complete and documented

15.2 Phase 2 Complete When

✓ Audit logs created for all mutations
✓ Change history tracked
✓ Restore from previous version working
✓ Soft deletes functioning

15.3 Phase 3 Complete When

✓ Socket.io connected
✓ Real-time events broadcasting
✓ Multiple browsers sync in real-time
✓ Offline handling graceful

15.4 Phase 4 Complete When

✓ Dashboard stats loading
✓ CSV export working for 5000+ rows
✓ Full-text search functional
✓ Audit log viewer operational

15.5 Phase 5 Complete When

✓ Backend deployed to Railway/Render
✓ Health check endpoint responding
✓ All endpoints working on live URL
✓ Logs show normal operation
✓ No errors for 24 hours
✓ Load test passes (100+ concurrent users)

15.6 Performance KPIs

Metric	Target	Status
API response time	<500ms	—
DB query time	<200ms	—
Login time	<1s	—
Socket.io broadcast latency	<2s	—
CSV export (1000 rows)	<5s	—
Memory usage	<200MB	—
CPU usage under load	<50%	—
Database storage	<512MB	—

Ready to Build!

This comprehensive guide provides everything needed to implement SmartUzhavan v4 backend.

Next Steps:

1. Review this document thoroughly
2. Gather team and assign Phase 1 tasks
3. Start with project initialization
4. Follow phase-by-phase checklist
5. Deploy to production

Estimated timeline: 5 weeks with 1 developer
Total cost: $0/month (all free tiers)
Scalability: Handles 100+ concurrent users

Good luck! You've got this! ■